

Open Source LVC Simulation architecture

Rafael Rodrigues Pereira
Mackenzie Presbyterian University
São Paulo, Brazil
<https://orcid.org/0009-0007-6773-1202>

Luiza Ribeiro dos Santos
Mackenzie Presbyterian University
São Paulo, Brazil
<https://orcid.org/0009-0009-7756-3315>

Marcos Henrique Mansano Vogel
Mackenzie Presbyterian University
São Paulo, Brazil
<https://orcid.org/0009-0001-0431-714X>

Bruno da Silva Rodrigues
Mackenzie Presbyterian University
São Paulo, Brazil
<https://orcid.org/0000-0001-9567-4910>

Fabio Silva Lopes
Mackenzie Presbyterian University
São Paulo, Brazil
<https://orcid.org/0000-0001-8274-7682>

Fabio Kawaoka Takase
Mackenzie Presbyterian University
São Paulo, Brazil
<https://orcid.org/0009-0006-4187-796X>

Abstract—The inherent complexity of urban management in the context of Smart Cities demands advanced solutions for planning and decision support, aiming to increase situational awareness and resilience of urban ecosystems. This work proposes and validates an event-driven software architecture for a platform that integrates the Live, Virtual, and Constructive (LVC) domains over the Mackenzie Presbyterian University Campus Living Lab. Grounded on the Event Sourcing (ES) and Command Query Responsibility Segregation (CQRS) patterns, the architecture is built exclusively on Open Source technologies, overcoming the licensing and adoption barriers of traditional High Level Architecture (HLA) and Distributed Interactive Simulation (DIS) standards. The canonical interoperability contract is defined via Protocol Buffers and gRPC, with automated generation of Java, Python, and Rust stubs by the `proto-shared` module. The platform was materialized as six independent modules and exercised through Live adapters for AIS, SPTrans (OlhoVivo) and CGESP, a Virtual component with a Vaadin interface over the Higienópolis campus, and a Constructive component based on AirSim with trajectory planning supported by NVIDIA NIM and Ollama Gemma. Four integrated LVC scenarios (registration of areas and POIs, entity check-in/check-out, drone mission triggered by Live events, and alert auditing) were demonstrated over two Grafana observability dashboards. Results validate the architecture's technical feasibility along four dimensions: traceability, reproducibility, multi-language interoperability, and integration with real sources. As limitations, the statistical comparative validation between Live data and Virtual/Constructive models, and the evaluation of horizontal scalability under massive load, remain as future work.

Keywords—Digital Twins; LVC; Event-Driven Architecture; Event Sourcing; Open Source.

I. INTRODUCTION

The complexity of managing a city, or even a significant portion of it, represents a major challenge that requires support for planning, operation, monitoring, and control, with the goal of achieving greater situational awareness, performance, resilience, and predictability in the face of emerging challenges.

Originally used in industry for the development of complex products [1], digital twins have recently become an attractive alternative [2] in the pursuit of actionable situational awareness. As live-virtual models that replicate physical systems, including their surrounding environment and operational processes, digital twins establish a continuous flow of data between the physical and virtual worlds, thereby expanding understanding and enhancing the management of complex ecosystems [3].

The effective use of digital twins in urban environments faces several obstacles, including data collection, system connectivity, the handling of large volumes of information, and the integration and interoperability of different technologies and systems [2]. Another challenge lies in the visualization of scenarios and data in ways that enhance situational awareness for planning and decision-making.

Constructive simulation is also employed to enrich scenarios with rare and/or high-risk events in training campaigns and system testing activities [4], [5]. Such simulations can even model the behavior of groups of people [6]. These capabilities can likewise be applied to urban contexts involving scenarios of interest to civil defense and public security.

This work aims to address an urgent technological development need [7] by proposing a systems architecture for a platform that supports integration and interoperability among data acquisition systems, storage infrastructures, digital models, visualization solutions, and simulation tools. The core use case is the Mackenzie University campus, supporting activities conducted within the FCI (School of Computing and Informatics) Digital Twins research project and the FCI technological development and partnership program known as Campus Living Lab.

II. BACKGROUND AND RELATED WORK

Simulation, a modeling technique aimed at representing and reproducing the dynamic behavior of a real or conceptual system over time [8], is an essential tool for the analysis, projection, and optimization of systems. Its application provides a controlled environment for experimentation and decision-making, making it fundamental for solving real-world problems. With technological advances, simulation has evolved from a predictive tool into a broader environment for experimentation and training, seeking greater realism and integration capabilities. This pursuit of interoperability in these models is important for meeting requirements regarding reusability and scalability, and, ultimately, the expected outcomes of training and capacity-building activities [9].

In complex urban contexts, such as those of Smart Cities, simulation materializes through the concept of the Digital Twin. A Digital Twin acts as a faithful and dynamic virtual representation of a physical system (a city or an urban subsystem), integrated with real-world data. This virtual artifact enables forecasting, analysis, and, crucially, scenario simulation, supporting urban decision-making through the combination of real and virtual environments. Given the increasing complexity of contemporary systems, which require the validation of models against real-world data and the simulation of dangerous or impractical scenarios in the physical world, traditional simulation has become insufficient. This need has driven the development of more sophisticated approaches, such as the LVC paradigm. LVC represents a broader objective within the field of Modeling and Simulation (M&S), aiming to create an integrated and unified simulation architecture [10]. LVC refers to the integration of three types of simulation resources:

- *Live*: Real people operating real systems.
- *Virtual*: Real people operating simulated systems.
- *Constructive*: Simulated people operating simulated systems.

This integration aims to create a cohesive and interactive environment for simulations, although achieving LVC interoperability remains a considerable challenge for the Modeling and Simulation (M&S) community [9].

The LVC paradigm seeks to maximize the realism and utility of simulation by allowing these three components to coexist and interact within a single simulated scenario. Such integration enables a seamless transition across different testing phases [5]. In the context of Smart Cities, LVC integration is particularly powerful because it enables the validation of *Virtual* models against *Live* reality while allowing the safe exploration of high-risk *Constructive* scenarios, such as disasters or demand surges. Among its many applications, LVC has been employed for the modeling and simulation of complex combat scenarios

[9], the testing, evaluation, and training of cyber operations [11], and the assurance of safety and security for complex autonomous vehicles through the evaluation of responses to diverse situations in virtually constructed worlds [12].

Interoperability among the components is crucial to the success of an LVC simulation, as it must ensure accurate data flow and consistency among data originating from different LVC sources and delivered to the client [5].

To support these integrated simulations, architectures such as the HLA (IEEE Std 1516-2025) and the DIS (<https://open-dis.org/>) are frequently employed, representing important milestones in the history of distributed simulation architectures [10]. HLA provides the structural foundation for simulation interoperability [13], with the objective of enabling simulation reuse and interoperability. DIS, in turn, is a communication protocol focused on real-time data exchange among simulators [10]. Modularity is a key factor in this environment, as it enables the separation of functionalities with different development and evaluation cycles, thereby facilitating the maintenance and evolution of the LVC environment [5].

However, HLA and DIS present significant limitations in the context of academic research and open development. HLA requires the use of a *Run-Time Infrastructure* (RTI), which is often proprietary and associated with substantial licensing costs, creating a significant barrier to entry for laboratories and universities [10]. Furthermore, the steep learning curve, dependence on highly specialized tools, and limited integration with the modern open-source software ecosystem restrict the agile adoption of these standards in research projects [7]. This reality motivates the search for alternative architectures capable of providing comparable interoperability at lower cost, with greater accessibility and full compatibility with *Open Source* technologies.

Given this challenge, attention shifts toward software architectures capable of supporting high-performance interoperability and massive scalability. Such environments require architectures that are reactive, in the sense of being *message-driven*, elastic, resilient, and responsive, in accordance with the principles of the *Reactive Manifesto* [14].

Within this context of distributed and reactive systems, architectural patterns such as ES and CQRS emerge as robust solutions. CQRS [15] proposes a separation of responsibilities by distinguishing command processing (write requests that modify system state) from query processing (read requests that do not modify state) [16]. ES, in turn, is a fundamental pattern frequently used alongside CQRS [14], in which the application state is stored as a sequence of immutable events [17]. Together, these patterns provide a foundation for the development of highly available and scalable systems.

To enable efficient, multilingual communication among the

heterogeneous components of the platform, *Protocol Buffers* (ProtoBuf) is adopted as the Interface Definition Language (IDL), while gRPC serves as the high-performance remote communication protocol with binary payloads [18]. Developed by Google, ProtoBuf is a language- and platform-agnostic binary serialization mechanism. From a `.proto` contract file, it automatically generates serialization and deserialization code in multiple programming languages, ensuring a unified and strongly typed data contract across services. Built on top of HTTP/2, gRPC provides low-latency synchronous and asynchronous communication with native support for bidirectional *streaming*. The ProtoBuf + gRPC combination establishes the canonical interoperability contract of the LVC platform, enabling components implemented in different programming languages (such as Java, Python, and Rust) to communicate transparently, in a loosely coupled manner, and with minimal network overhead.

Additionally, the architecture relies on well-established reactive design patterns to enhance resilience and responsiveness in distributed environments. The *Hexagonal Architecture* (ports & adapters) pattern enables a clear separation between business logic and infrastructure concerns, facilitating testing and incremental evolution. Combined with the principles of the *Reactive Manifesto*, these patterns guide the platform toward becoming *message-driven*, resilient, and responsive, even under high workloads.

III. ARCHITECTURE

A. Architecture and Design Decisions

The software architecture is the central artifact of this work, and its definition constitutes the second stage of the research. The starting point is the set of challenges identified in the literature review: the integration of heterogeneous data sources, interoperability among components implemented in different programming languages, and the licensing and integration barriers associated with traditional distributed simulation standards (HLA and DIS). The procedure consists of deriving solution requirements from these challenges and, based on them, evaluating candidate communication paradigms and architectural patterns, with a preference for open and accessible technologies. The scope is limited to a technical feasibility study conducted at a proof-of-concept scale.

The proposed solution must satisfy a concise set of requirements: a single, strongly typed data contract shared by all components, from which consistent interfaces across different programming languages can be derived, enabling each module to integrate regardless of its implementation technology; asynchronous and loosely coupled communication among the *Live*, *Virtual*, and *Constructive* domains, with real-time event consumption; the ability to compare the *Virtual* and *Live* domains

in order to assess model fidelity; and complete, extensible, and accessible documentation available to any developer.

The evaluation of the resulting architecture will be conducted across four dimensions, aligned with the validation stage of the research: traceability, reproducibility, multi-language interoperability, and ease of integration with real-world data sources, all exercised through integrated LVC scenarios. Throughout the project, adherence to the reactive principles defined by the *Reactive Manifesto* will be pursued, with the goal of building a robust, scalable, and resilient system. The selected patterns and technologies, as well as the results of this evaluation, are presented in the Results and Discussion section.

B. Tools and Data Sources

Demonstrating technical feasibility requires evidence that the architecture can accommodate heterogeneous data sources and distinct simulators. For this reason, the work is not focused on a single application scenario, but rather on the integration of diverse data sources (public APIs, *web scraping*, open geospatial data, and simulators) capable of exercising the interoperability contract and the reactive behavior of the proposed architecture under real-world inputs.

All data used in this study are collected from public and freely accessible sources, aligning with the strictly *Open Source* nature of the architecture. Access to the OlhoVivo (SPTrans) API is provided through an open and free registration process; access to the AIS API via aisfriends.com is also freely available; CGESP data are obtained from public information made available by the City of São Paulo, with non-abusive query frequencies; and OpenStreetMap geospatial data are freely available for use.

To support this approach, the following tools were studied, each justified by the role it plays within the LVC lifecycle:

- *OpenStreetMap* [19]: an open geospatial database used to define patrol areas and points of interest on the Higienópolis campus, serving as input for the *Virtual* component.
- *AirSim* [20]: a drone flight simulator operating in a photo-realistic urban environment, employed as the *Constructive* basis for simulated missions within the virtual world.
- *AIS (Automatic Identification System)* via aisfriends.com [21]: a public real-time vessel tracking API, used as a *Live* source of maritime mobility data.
- *OlhoVivo* (SPTrans) [22]: a public API providing bus location data in São Paulo, used as a *Live* source of urban mobility information.
- *CGESP* [23]: a public portal maintained by the Municipality of São Paulo containing flood-related information, used as a *Live* source of civil defense events through *web scraping*.

The deliberate choice of multiple data sources and tools, rather than a single monolithic scenario, is itself part of the validation process: to demonstrate that the proposed architecture supports the *plug-and-play* integration of distinct types of data sources.

C. Deployed Architecture

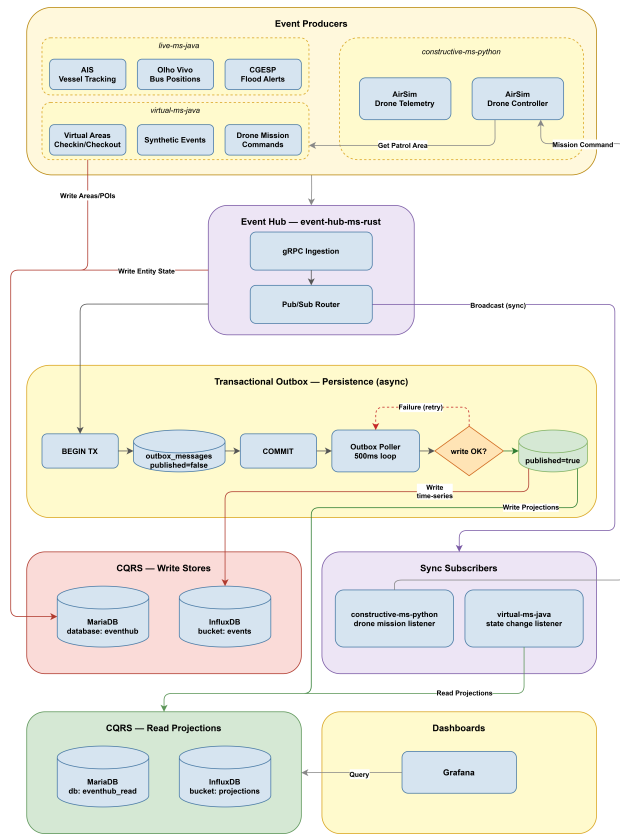


Figure 1. Event-driven LVC platform architecture.

The evaluation conducted during the architecture definition phase resulted in the adoption of an event-driven model. LVC systems are inherently reactive: events from the real world (*Live*) trigger updates in digital models (*Virtual*), which in turn may initiate simulations (*Constructive*); an *event-driven* architecture mirrors this dynamic while avoiding the temporal coupling and single points of failure associated with synchronous request-response interactions.

The LVC platform, whose architecture is presented in Figure 1, is composed of six independent modules, each implemented using the programming language and technology stack best suited to its domain, while being unified through a canonical communication contract based on *Protocol Buffers* and gRPC.

This polyglot approach is deliberate: rather than forcing all components into a single programming language, as is common in typical HLA implementations that require specific C++ or Java bindings, each module leverages the most productive ecosystem for its particular function.

The *proto-shared* module acts as the single source of contractual truth, automating the generation of client and server code in Java (via Maven), Python (via the *ascopes protobuf-maven-plugin*), and Rust (via a dedicated Docker image using *protoc-gen-prost* and *protoc-gen-tonic*), thereby eliminating the barriers associated with multi-language integration.

This is a *contract-first* approach: the *.proto* file serves as the single source of truth from which client and server stubs are generated for all three programming languages, ensuring that contracts remain consistently synchronized across the entire platform. Table I summarizes the technologies adopted by each component.

The last two projects, *proto-shared* and *infra-local-shared*, are cross-cutting in nature: the former operates at *build* time (stub generation), while the latter provides the local execution infrastructure; therefore, they are not represented as *runtime* nodes in Figure 1.

The *event-driven* communication flow among the components follows the CQRS paradigm, with an explicit segregation between the write (*Write*) and read (*Read*) paths:

- **CQRS-Write:** The *Live* (*live-ms-java*) and *Constructive* (*constructive-ms-python*) components publish events to the *Event Hub* (*event-hub-ms-rust*) via gRPC. The *Event Hub* persists events in InfluxDB (*time-series*) and MariaDB (*relational outbox*), ensuring atomicity through the *Transactional Outbox Pattern*. The *Virtual* component (*virtual-ms-java*) writes patrol areas and Points of Interest (POIs) directly to MariaDB (*CQRS-Write*).
- **CQRS-Read:** The *Virtual* component queries the projection database (*eventhub_read*) in MariaDB and projection buckets in InfluxDB to obtain the materialized state of the digital twins. Grafana consumes data from InfluxDB (*projection buckets*) and MariaDB (*eventhub_read*) to generate real-time dashboards.
- **Inter-component Communication:** The *Constructive* component subscribes to drone mission events in the *Event Hub* and queries the *Virtual* component via gRPC to obtain patrol areas. In turn, the *Virtual* component subscribes to events from the *Event Hub* to detect state changes and publishes new events back to the event bus.

The source code for all components has been fully published as *Open Source* in a GitHub repository (<https://github.com/>)

Table I
OVERVIEW OF THE COMPONENTS OF THE LVC PLATFORM AND THEIR RESPECTIVE TECHNOLOGY STACKS.

Project	Language	Framework	Key Technologies
<i>Runtime Components</i>			
constructive-ms-python	Python 3.11+	AirSim + FastAPI	NVIDIA NIM APIs, Ollama Gemma 4, gRPC, Protocol Buffers
event-hub-ms-rust	Rust	gRPC + HTTP/2	Bidirectional Pub/Sub, MariaDB/InfluxDB, Docker, Outbox Pattern
live-ms-java	Java 25	Spring Boot 4	Hexagonal Architecture, Resilience4j, Virtual Threads, Feign, Jsoup
virtual-ms-java	Java 25	Spring Boot 4 + Vaadin 25.1	gRPC, MariaDB, InfluxDB, CQRS Write/Read
<i>Support Projects (build-time and infrastructure)</i>			
proto-shared	N/A	Apache Maven + Docker	Protocol Buffers 4.34, gRPC 1.79, ascpes protobuf-maven-plugin (Java + Python), Docker codegen (Rust)
infra-local-shared	N/A	Docker Compose	InfluxDB 2.7, MariaDB 11.4, HashiCorp Vault 1.17, Grafana

tcc-modern-lvc-arch) and documented on Deepwiki [24] (<https://deepwiki.com/tcc-modern-lvc-arch>). Deepwiki automatically generates navigable and searchable documentation directly from the repository, keeping it synchronized with the code-base and freely accessible to any developer, thereby fulfilling the requirement for comprehensive, extensible, and accessible documentation.

D. Component Implementation

The implementation follows a modular and polyglot approach. Each component was designed to be developed, tested, and deployed independently, an essential property for enabling the plug-and-play integration of new Digital Twins and simulators without recompiling or reconfiguring the central bus.

1) *Live Component — live-ms-java*: The *Live* component is implemented as a Spring Boot 4 (Java 25) microservice using a *Hexagonal Architecture* (ports & adapters) approach. It integrates three real-time geospatial data sources:

- AIS: vessel tracking through the aisfriends.com API [21], integrated using Feign Client together with a Resilience4j Circuit Breaker.
- OlhoVivo (SPTrans): bus location data provided through the proprietary SPTrans API, featuring lazy authentication and session management.
- CGESP: flood monitoring (flood points) obtained through web scraping (Jsoup) with structured HTML data parsing.

Each adapter implements a scheduled polling pattern (@Scheduled) and publishes events to the *Event Hub* via gRPC. The high concurrency resulting from multiple simultaneous connections is supported by *Virtual Threads* (Java 25), providing low overhead.

All implemented adapters consume exclusively public data sources. Access to the OlhoVivo (SPTrans) API is provided

through an open and free registration process, and the data are made available as public information with no possibility of commercialization. CGESP data are obtained from public information sources provided by the São Paulo City Administration, using a non-abusive polling frequency.

2) *Constructive Component — AirSim with AI*: The *Constructive* component is implemented in Python and integrates the AirSim simulator for drone flight simulation in an urban environment. Trajectory planning and drone decision-making leverage the free NVIDIA NIM (NVIDIA Inference Microservices) APIs, which provide pre-trained AI models for computer vision and trajectory control at no cost, avoiding the performance overhead of running computationally intensive inference on the local machine. As a local offline alternative, the project also integrates the Ollama Gemma 4 model (gemma4:e2b) for natural language inference in mission planning. The simulator publishes drone position, sensor, and status events via gRPC to the *Event Hub*.

3) *Virtual Component — Digital Twin*: The *Virtual* component is implemented as a Spring Boot 4 (Java 25) microservice with a Vaadin 25.1 user interface, materializing the Digital Twin of the Higienópolis campus. The module manages patrol areas (georeferenced zones for the simulated drone), Points of Interest (POIs), and the state of urban entities monitored by the platform. CQRS is materialized through two segregated paths: the *write path* persists commands and state updates in the eventhub database (MariaDB), while the *read path* consumes materialized projections from the eventhub_read database (MariaDB) and projection buckets in InfluxDB to feed the Vaadin interface and Grafana dashboards. The *Virtual* component also subscribes to events from the *Event Hub* to detect state changes in the *Live* and *Constructive* components, publishing new events back to the event bus as the Digital Twin

state is updated.

4) Event Bus and Persistence:

- **Event Hub (Rust + gRPC):** materializes the asynchronous and decoupled event bus, implementing bidirectional *Pub/Sub* through gRPC streaming and Protocol Buffers as the data contract. It implements the *Transactional Outbox* pattern to ensure reliable event delivery: every received message is atomically persisted in MariaDB (outbox_messages table) and asynchronously published to subscribers, with automatic retries in case of network failures. Subscriber consumption occurs through gRPC streaming, without polling, supporting the real-time updates observed in the dashboards.
- **Containerization:** Docker Compose with the shared-infra-network network interconnecting the *Event Hub*, live-ms-java, virtual-ms-java, and the AirSim simulator, ensuring portability and reproducibility.
- **Event Persistence (CQRS-Write):** InfluxDB serves as the time-series database for storing LVC events, with configurable retention policies by bucket (events for writes and projections for reads). MariaDB uses two segregated databases: eventhub (CQRS-Write) for entity, state, and outbox records, and eventhub_read (CQRS-Read) for materialized projections consumed by Grafana.
- **Vault:** HashiCorp Vault is used for centralized secrets management (API tokens, credentials), together with the .env pattern for local configuration.

E. Geospatial Data Sources

The urban environment simulated by the *Constructive* component and modeled by the *Virtual* Digital Twin uses geospatial data from OpenStreetMap. Figure 2 presents the polygon corresponding to the Mackenzie University campus area, which is used as the patrol zone for the simulated drone and as the region of interest for virtual-ms-java.

F. LVC Interaction Scenarios

The integrated platform was exercised through scenarios that connect the three LVC domains within a cohesive workflow, illustrating how real-world data (*Live*), digital models (*Virtual*), and simulations (*Constructive*) interoperate over the *event-driven* bus.

1) **Registration of Areas and Points of Interest (Virtual):** The starting point of the cycle is the registration, through the Vaadin SPA of virtual-ms-java, of georeferenced areas (patrol zones) and Points of Interest (POIs) within the Higienópolis campus. These artifacts become part of the Digital Twin's state: areas serve as monitoring regions for *Live* entities, while POIs act as potential destinations for *Constructive* missions. Figure 3 presents the area registration interface.

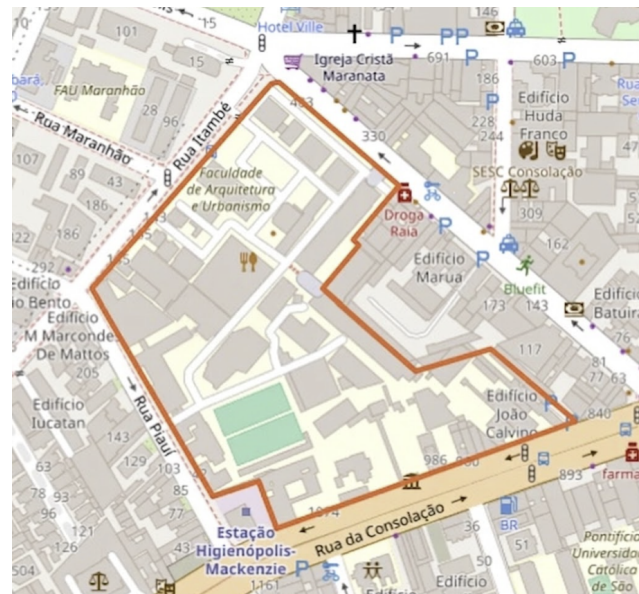


Figure 2. Polygon of the Higienópolis campus extracted from OpenStreetMap.

Name	Type	Monitors	Action	Status	Actions
Mackenzie Campus	POLYGON	DRONE	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete
Rua da Consolação	CORRIDOR	BUS	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete
Consolação Neighborhood	POLYGON	BUS	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete
Caio Prado C/B Stop Area	CIRCLE	BUS	MISSION_TRIGGER	Active	Edit Deactivate Delete
Santos Port	CIRCLE	VESSEL	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete

Figure 3. CRUD operations for patrol areas in the virtual-ms-java SPA

2) **Live Ingestion and Check-in/Check-out Events:** The *Live* adapters (AIS, OlhoVivo, and CGESP) continuously publish real-world entity position events to the *Event Hub*. The *Virtual* component subscribes to this stream and, upon each position update (the trigger), performs a geometric verification to determine whether the entity has become contained within the polygon of a registered area or has left it. A transition into the area generates a *check-in* event, while a transition out of the area generates a *check-out* event, each associating the entity with the area in the Digital Twin's state. This entity-area matching mechanism is how *Live* data sources become observable within the Digital Twin, without direct coupling between the adapter and the model.

3) **Drone Mission (Constructive) Triggered by a Live Event:** A drone mission is defined as the *Constructive* action of moving the simulated drone to the coordinates of a POI for visual inspection. The trigger occurs when a *Live* entity arrives at a POI (for example, a bus registered at the *Caio Prado C/B* bus stop through the OlhoVivo API), leading the *Virtual* component

to publish an *entity-at-POI* event. The `constructive-ms-python` component subscribes to this event, queries the *Virtual* component via gRPC to obtain the mission coordinates, and initiates the simulated flight in AirSim, with trajectory planning supported by NVIDIA NIM (or by a local offline alternative using Ollama Gemma). Upon reaching the destination, the drone captures photographs and publishes the resulting artifacts back to the event bus, where they are audited. The same mechanism can also be triggered by synthetic events generated through the SPA (Figure 4), enabling controlled reproduction of the scenario.

The screenshot shows a web interface titled 'Virtual Areas'. On the left, there's a sidebar with 'Areas', 'Points of Interest', 'Simulation', and 'Monitoring'. The 'Simulation' tab is active. The main area has two sections. The first section is 'Ping Virtual Move Event' with fields for 'Entity ID' (e.g., vehicle-001), 'Entity Type' (BUS), 'Latitude', 'Longitude', 'Altitude (in optional)', and 'LVC Origin' (VIRTUAL). There's a 'Send Move Event' button. The second section is 'Simulate Virtual Flood Event' with fields for 'Area', 'Severity' (moderado), 'Water Level (cm)' (50), and 'LVC Origin' (VIRTUAL). There's a 'Simulate Flood' button.

Figure 4. Synthetic event triggered from the `virtual-ms-java` SPA, used to simulate the activation of *Constructive* scenarios.

4) *Flooding Events and Alert Auditing*: A flooding event is defined as a *Live* civil defense event: through web scraping, the CGESP adapter detects a flooding location reported by the City Administration (the trigger) and publishes it as an event on the bus, which feeds the Digital Twin and may trigger a verification mission by the *Constructive* drone. Since no flooding events were reported by CGESP during the validation period, the scenario remains architecturally supported and reproducible through synthetic triggering via the SPA (Subsection III-F3). In parallel, collision (*crash*) events occurring during the simulated flight are captured by the *Event Hub* and enter the auditing pipeline, contributing to the complete observability framework described in Subsection III-G.

G. Validation and Observability

The consolidation of the developed components was operationally exercised across three fronts:

- **Event Bus (*Event Hub*)**: the *Event Hub* application (developed in Rust), which is central to the architecture's *Publish/Subscribe* pattern, was modified and containerized using Docker, satisfying the requirements of Reproducibility and ease of deployment. Event persistence was implemented on four segregated data stores: MariaDB `eventhub` (CQRS-Write) and `eventhub_read` (CQRS-Read), in addition to InfluxDB buckets events

(write) and projections (read for Grafana), eliminating the dependency on Docker volumes for durability.

- **proto-shared Module**: to ensure interoperability among the various LVC components, the `proto-shared` module was developed and validated. This artifact, structured as an Apache Maven project and containing no proprietary source code (zero-source-code artifact), automates the compilation of the IDL (*Protocol Buffers*), generating communication stubs in Java, Python, and Rust from a single canonical `.proto` contract. Generation for Java and Python is performed through the `ascopes protobuf-maven-plugin` [25] integrated with Apache Maven, while Rust stubs are produced by a dedicated Docker image using `protoc-gen-prost` and `protoc-gen-tonic`, ensuring standardization and consistency of the integration interfaces across LVC components.
- **Live Ingestion Adapters**: the three adapters of `live-ms-java` (*AisAdapter*, *OlhoVivoAdapter*, and *CgespAdapter*) were implemented and operationally verified, demonstrating the ability to ingest geospatial data from heterogeneous real-world sources (AIS, SPTrans, and CGESP) and publish them as standardized events to the *Event Hub*, thereby closing the *Live-to-Platform* integration cycle.

Platform observability is provided by two Grafana dashboards, fed through the CQRS-Read path (the InfluxDB `projections` bucket and the MariaDB `eventhub_read` database), which consolidate platform operations into complementary monitoring panels.

Two Grafana dashboards (Figure 5) consolidate platform observability through the CQRS-Read path. The upper dashboard provides the operational view (entities monitored by area, *check-in* and *check-out* counts, an interactive map with entity history, and an audit log), highlighting the *Live-to-Virtual* flow and the read-side segregation enabled by CQRS. The lower dashboard focuses on alert auditing (flooding events reported by CGESP and collisions detected during drone simulation) and the observability of photographs captured by the *Constructive* component at POI coordinates (illustrated by the *Caio Prado C/B* bus stop), thereby closing the *Live-Virtual-Constructive-Observability* cycle within a single control plane.

IV. RESULTS AND DISCUSSION

The results obtained validate the technical feasibility of the proposed architecture across four fundamental dimensions. First, traceability: the audit dashboard (Figure 5, lower panel) reconstructs the complete sequence of events that triggers each *Constructive* mission, from the *check-in* of a *Live* entity at a POI to the publication of photographs captured by the

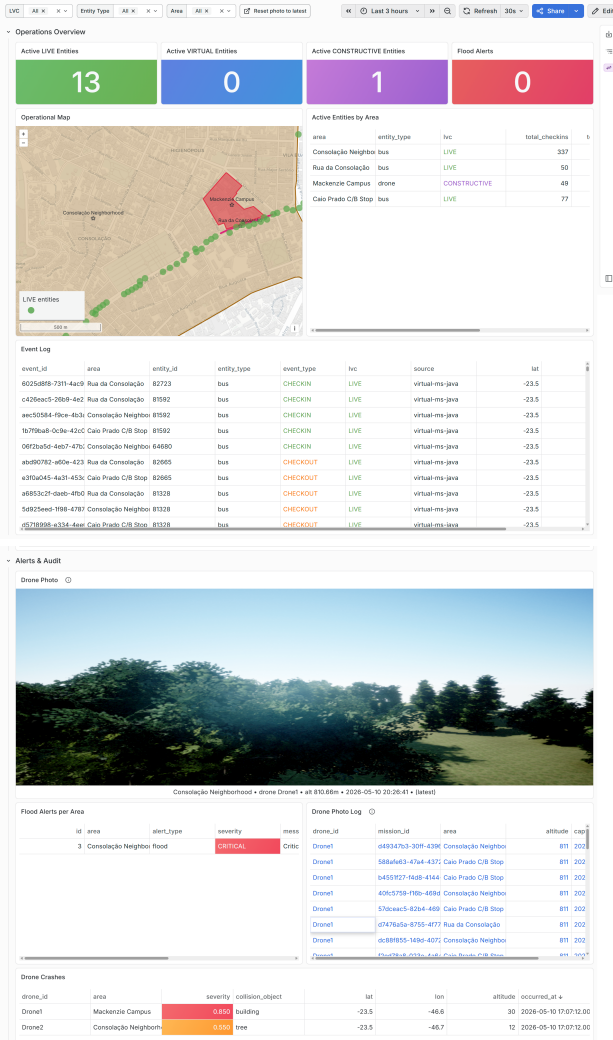


Figure 5. Grafana observability dashboards for the LVC platform. **Top panel:** operational view showing entities by area, check-in/check-out events, historical map, and audit log. **Bottom panel:** alert auditing (CGESP flood alerts and simulated drone collisions) and observability of photos captured by the Constructive service.

drone. Each event is immutably persisted in InfluxDB and MariaDB (through the *Transactional Outbox*) and can be referenced by timestamp and identifier, enabling full scenario replay as prescribed by the *Event Sourcing* pattern. Second, reproducibility: the complete containerization of the *Event Hub* and the microservices ensures that any researcher can reproduce the execution environment with a single command. Third, multi-language interoperability: the *proto-shared* module successfully demonstrated the generation of communication stubs in Java, Python, and Rust from a single *.proto* contract,

confirming that components implemented in heterogeneous programming languages can integrate seamlessly through strongly typed interfaces. Fourth, integration with the real world: the three adapters of *live-ms-java* demonstrated the ability to ingest geospatial data from real-world sources (AIS, SPTrans, and CGESP) and publish them as standardized events on the bus, thereby closing the *Live-to-Platform* integration cycle. The diversity of integration mechanisms involved (authenticated REST APIs, web scraping, and registration-based APIs) under a single canonical contract provides empirical evidence of the architectural integration flexibility envisioned by Objective 4.

The adopted patterns also distinguish themselves from traditional alternatives in several key aspects. *Event Sourcing*, unlike the HLA federation model where simulation state is volatile and distributed among federates, provides an immutable and centralized event log that simplifies debugging and auditing. The platform's reactivity operates at the architectural level (asynchronous event bus and gRPC streaming) rather than at the framework level. Spring Boot WebFlux and Project Reactor were intentionally not adopted because the gRPC ecosystem already provides native non-blocking I/O over HTTP/2, and *Virtual Threads* (Java 25) offer scalable concurrency using a simple imperative programming model. This approach avoids the complexity of reactive streams (Mono/Flux) while simplifying debugging. Additionally, the use of *Scoped Values* (JEP 506) for context propagation across *Virtual Threads* replaces *ThreadLocal*, which is problematic in highly concurrent environments.

These findings directly align with the literature. The proposed architecture addresses the limitations of HLA and DIS identified by Gao et al. [10] and Dani et al. [7], such as licensing costs, steep learning curves, and limited integration with modern software ecosystems, by replacing them with a stack of mature and widely adopted *Open Source* technologies. *Event Sourcing*, as argued by Debski et al. [14] and Overeem, Spoor and Jansen [17], provides traceability and reproducibility capabilities that are absent in the HLA federation model, where state is volatile and distributed. CQRS, in turn, enables a clear separation of concerns between high-frequency data ingestion (*Live/Constructive*) and low-latency queries (dashboards/*Virtual*), aligning with the principles of the *Reactive Manifesto* [14].

The platform's *Open Source* nature constitutes a contribution in itself. By making both the source code and documentation fully available through Deepwiki, this work provides an accessible, zero-cost alternative to traditional proprietary architectures, removing the financial barriers that have historically limited LVC simulation research to institutions capable of funding RTI licensing costs.

Nevertheless, limitations inherent to the scope of this tech-

nical feasibility study must be acknowledged. Comparative statistical validation between *Live* data and *Virtual/Constructive* models, which is essential for demonstrating Digital Twin fidelity, remains future work and depends on the complete integration of the simulation cycle. Similarly, all experiments were conducted on a single machine, and the horizontal scalability of the event bus under massive event loads was not evaluated. These limitations do not invalidate the findings, but rather define their scope: what has been demonstrated is architectural and integration feasibility, not predictive model accuracy or performance under extreme load conditions.

V. CONCLUSIONS

This work presented the design, implementation, and validation of an event-driven architecture for an LVC simulation platform applied to the *Smart Cities* domain using exclusively *Open Source* technologies. The four objectives established were achieved: the literature review provided the foundation for selecting the *Event Sourcing*, CQRS, and reactive architecture patterns; the architecture was defined through a canonical contract based on *Protocol Buffers* and gRPC; the implementation was realized through six independent modules (proto-shared, event-hub-ms-rust, live-ms-java, constructive-ms-python, virtual-ms-java, and infra-local-shared); and the validation exercised the contract across multiple real-world data sources and a simulation environment.

The platform was not conceived as an instance of a specific urban scenario, but rather as an extensible foundation for integrating diverse data sources, including public APIs, web scraping, open geospatial datasets, and simulators, all publicly available and free of charge. The LVC interaction scenarios presented in Subsection III-F complete this cycle: the registration of areas and POIs in the *Virtual* Digital Twin, the continuous ingestion of *Live* entities with area-based *check-in* and *check-out* events, the triggering of *Constructive* drone missions from real-world events (such as a bus arriving at the *Caio Prado C/B* bus stop) with photo capture and auditing, and the propagation of drone collision and flooding alerts (when reported by CGESP) into the Grafana observability layer.

This work makes four primary contributions: (i) an *Open Source*, event-driven architecture for LVC simulation that replaces HLA/DIS without sacrificing interoperability, directly addressing the cost and complexity barriers identified by Gao et al. [10] and Dani et al. [7]; (ii) the proto-shared module as a single source of contractual truth, validating the automated generation of multi-language stubs (Java, Python, and Rust) from a single .proto definition; (iii) the integration of three real-world and publicly accessible *Live* data sources

(AIS, SPTrans, and CGESP) as empirical evidence of heterogeneous data ingestion under a canonical contract; and (iv) concrete LVC scenarios that exercise the complete *Live*, *Virtual*, *Constructive*, and Observability cycle end-to-end within the Mackenzie University campus.

The technical feasibility study, which constituted the primary objective of this work, has therefore been successfully accomplished. The results demonstrate that LVC interoperability can be achieved using accessible tools without sacrificing performance, traceability, or reproducibility. The platform establishes the foundations for an open and extensible urban simulation ecosystem and represents a significant step toward democratizing Digital Twin and simulation research in the context of *Smart Cities*. As the study integrates different concepts and technologies, including Open Source, event-driven architecture for LVC simulation, three real-world and publicly accessible live data sources, and the proto-shared module as a single source of contractual truth, one limitation of this work was the difficulty in identifying similar implementations for comparison with the proposed solution. Future work includes comparative statistical validation between *Live* data and *Virtual/Constructive* models, as well as the evaluation of event bus horizontal scalability under massive event loads.

ACKNOWLEDGEMENTS

This research was supported by the MackPesquisa Fund through grant/process No. 2024.FCI.SPP.0005 and was conducted at MackLeaps facilities, a Research Laboratory of the School of Computing and Informatics.

STATEMENT ON THE USE OF ARTIFICIAL INTELLIGENCE

We hereby declare that Generative Artificial Intelligence tools were used in this study during the following stages: NVIDIA NIM was used for the trajectory planning for the drone mission during the data collection stage. All generated content was critically reviewed and validated by the authors, who assume full responsibility for the originality and accuracy of the final text, in accordance with CNPq Ordinance No. 2,664/2026.

REFERENCES

- [1] M. Grieves, "Digital Twin: Manufacturing Excellence through Virtual Factory Replication," Tech. Rep., 2014. [Online]. Available: <https://www.3ds.com/fileadmin/PRODUCTS-SERVICES/DELMIA/PDF/Whitepaper/DELMIA-APRISO-Digital-Twin-Whitepaper.pdf>
- [2] S. Mazzetto, "A Review of Urban Digital Twins Integration, Challenges, and Future Directions in Smart City Development," *Sustainability*, vol. 16, no. 19, p. 8337, Sep. 2024. [Online]. Available: <https://www.mdpi.com/2071-1050/16/19/8337>
- [3] Z. Lv, W.-L. Shang, and M. Guizani, "Impact of Digital Twins and Metaverse on Cities: History, Current Situation, and Application Perspectives," *Applied Sciences*, vol. 12, no. 24, p. 12820, Dec. 2022. [Online]. Available: <https://www.mdpi.com/2076-3417/12/24/12820>

- [4] J. R. Noseworthy, "The Test and Training Enabling Architecture (TENA) Supporting the Decentralized Development of Distributed Applications and LVC Simulations," in *2008 12th IEEE/ACM International Symposium on Distributed Simulation and Real-Time Applications*, 2008, pp. 259–268.
- [5] E. Theunissen and T. Kotegawa, "Applying LVC to testing and evaluation of DAA systems," in *2017 IEEE/AIAA 36th Digital Avionics Systems Conference (DASC)*. St. Petersburg, FL: IEEE, Sep. 2017, pp. 1–7. [Online]. Available: <http://ieeexplore.ieee.org/document/8102089/>
- [6] M. Loper and V. Sitterle, "Evolving Lvc To Include Evaluation Of Human-Ai Teaming Dynamics," in *2023 Winter Simulation Conference (WSC)*. San Antonio, TX, USA: IEEE, Dec. 2023, pp. 2506–2517. [Online]. Available: <https://ieeexplore.ieee.org/document/10408209/>
- [7] A. A. H. Dani, S. H. Supangkat, F. F. Lubis, I. G. B. B. Nugraha, R. Kinanda, and I. Rizkia, "Development of a Smart City Platform Based on Digital Twin Technology for Monitoring and Supporting Decision-Making," *Sustainability*, vol. 15, no. 18, p. 14002, Sep. 2023. [Online]. Available: <https://www.mdpi.com/2071-1050/15/18/14002>
- [8] J. Banks, Ed., *Handbook of Simulation: Principles, Methodology, Advances, Applications, and Practice*, 1st ed. Wiley, Aug. 1998. [Online]. Available: <https://onlinelibrary.wiley.com/doi/book/10.1002/9780470172445>
- [9] K. Kim, T. Park, J. Pastrana, M. Marin, E. A. Cortes, L. C. Rabelo, and G. Lee, "Modeling of complex scenarios using LVC simulation," in *Proceedings of the Winter Simulation Conference 2014*. Savannah, GA, USA: IEEE, Dec. 2014, pp. 2931–2941. [Online]. Available: <http://ieeexplore.ieee.org/document/7020133/>
- [10] Y. Gao, Y. Zhang, X. Zhou, and H. Lu, "Overview of Simulation Architectures Supporting Live Virtual Constructive (LVC) Integrated Training," in *2021 6th International Conference on Control, Robotics and Cybernetics (CRC)*. Shanghai, China: IEEE, Oct. 2021, pp. 333–338. [Online]. Available: <https://ieeexplore.ieee.org/document/9620135/>
- [11] M. Varshney, K. Pickett, and R. Bagrodia, "A Live-Virtual-Constructive (LVC) framework for cyber operations test, evaluation and training," in *2011 - MILCOM 2011 Military Communications Conference*, 2011, pp. 1387–1392.
- [12] B. Potteiger, T. Dignan, A. Mills, E. Pavelka, C. Frey, B. Nathan, M. Dagne, V. Garibaldi, and B. Otter, "Live Virtual Constructive Environment for Assuring the Safety and Security of Complex Autonomous Vehicles," in *2023 IEEE International Conference on Assured Autonomy (ICAA)*. Laurel, MD, USA: IEEE, Jun. 2023, pp. 53–56. [Online]. Available: <https://ieeexplore.ieee.org/document/10207574/>
- [13] J. Dahmann, "High Level Architecture for simulation," in *Proceedings First International Workshop on Distributed Interactive Simulation and Real Time Applications*. Eilat, Israel: IEEE Comput. Soc. Press, 1997, pp. 9–14. [Online]. Available: <http://ieeexplore.ieee.org/document/568652/>
- [14] A. Debski, B. Szczepanik, M. Malawski, S. Spahr, and D. Muthig, "A Scalable, Reactive Architecture for Cloud Applications," *IEEE Software*, vol. 35, no. 2, pp. 62–71, Mar. 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/7950867/>
- [15] Z. Long, "Improvement and Implementation of a High Performance CQRS Architecture," in *2017 International Conference on Robots & Intelligent System (ICRIS)*. Huai An City, China: IEEE, Oct. 2017, pp. 170–173. [Online]. Available: <http://ieeexplore.ieee.org/document/8101372/>
- [16] M. Ljubicavljević, N. Turajlić, M. Petrović, F. Furtula, and V. Belča, "A Comparison of Event Sourcing Implementations: Using Apache Kafka Streams APIs and Microsoft Service Fabric Actor Services," in *2024 23rd International Symposium INFOTEH-JAHORINA (INFOTEH)*. East Sarajevo, Bosnia and Herzegovina: IEEE, Mar. 2024, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/10495980/>
- [17] M. Overeem, M. Spoor, and S. Jansen, "The dark side of event sourcing: Managing data conversion," in *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. Klagenfurt, Austria: IEEE, Feb. 2017, pp. 193–204. [Online]. Available: <http://ieeexplore.ieee.org/document/7884621/>
- [18] F. Gerha Barbosa Otaviano Da Silva, M. Guido De Oliveira, M. H. M. Vogel, F. K. Takase, B. L. S. Lima, and L. A. M. Saito, "Comparing MQTT and gRPC Protocols Performance in a IoT Smart Campus Cybersecurity Scenario," in *2026 IEEE 17th Latin America Symposium on Circuits and System (LASCAS)*. Arequipa, Peru: IEEE, Feb. 2026, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/11457148/>
- [19] OpenStreetMap contributors, "OpenStreetMap," May 2026. [Online]. Available: <https://www.openstreetmap.org>
- [20] Microsoft Corporation, "AirSim," 2022. [Online]. Available: <https://github.com/microsoft/airsim>
- [21] AIS Friends, "AIS Friends API Documentation," 2025. [Online]. Available: <https://www.aisfriends.com/docs/api/v1>
- [22] São Paulo Transporte S/A, "API Olho Vivo: Guia de Referência." [Online]. Available: <https://www.sptrans.com.br/desenvolvedores/api-do-olho-vivo-guia-de-referencia>
- [23] Centro de Gerenciamento de Emergências (CGE), "Dados de Alagamentos e Monitoramento Meteorológico." [Online]. Available: <https://www.cgesp.org/v3/index.jsp>
- [24] Cognition Labs, "DeepWiki," 2025. [Online]. Available: <https://deepwiki.com>
- [25] A. Scopes, "Protobuf Maven Plugin," Apr. 2026. [Online]. Available: <https://scopes.github.io/protobuf-maven-plugin/index.html>